

DNV-BRAIN Hackathon 2025

Team 1

March 2025

1 Team members and background

Our team consists of 6 students, who are all currently in our second year of a M.Sc. in Cybernetics and Robotics. This far, none of our mandatory courses have covered machine learning, however we've learnt to see its value and role in autonomous systems. This served as a decent foundation, but, as we discovered, did not flatten the learning curve any substantial amount. Luckily, one of our members, Magnus, has through a voluntary position at Ascend NTNU worked with perception, and therefore had relevant experience with Ultralytic's YOLO-model.

2 Overview of solution

During model-developing we focused our attention on two areas and split our group accordingly. The first group focused on finding and optimizing the best-performing YOLO-model, meanwhile the other group focused on other resources, such as helping the trained model with inference and filtering out bad detections.

2.1 Model Design

In the early stages, we explored alternative training models beyond YOLO11n (Khanam and Hussain 2024) and compared performance. After researching studies and testing models, we settled on YOLO12m (Tian et al. 2025), which achieved the best mAP50 score—our key metric. However, we frequently reconsidered this choice.

After selecting a model, we investigated hyperparameter tuning using assignment guidelines and external sources. Key parameters included multi-scale training, mixed precision, COCO-pretrained weights, epochs, and optimizers. Despite careful tuning of these parameters, mAP50 remained unchanged, likely due to a small dataset or limited epochs.

2.2 Model inference and post processing

As the purpose of this challenge is to develop models to investigate reliability and explainability of AI methods, a big part of our approach has been post-processing detections. With simple heuristics obtained from analysing the dataset and exploitation of full image resolution, we were able to remove a significant amount of both false- positives and negatives.

2.2.1 Image slicing

After analyzing the dataset, we observed that a big part of this task is to detect small objects. The dataset images are 1920×1088 px², while YOLO models prefer 640×640 px², causing information loss when downscaled. To mitigate this, we used image slicing with the SAHI framework during inference (Akyon et al. 2022).

SAHI works by moving a sliding window over the input image, with a user-controlled overlap between slices. After this implementation, the mAP-50 score increased significantly on the validation dataset.

2.2.2 Additional filtering

Noticing that all handout images were taken from the same spot on the "Hurtigruten" vessel, we filtered out predictions falling within the front beam's area

During validation, we noticed false positives over the horizon. To address this, we annotated the handout dataset with a "background" label for areas above sea level (e.g., horizon, docks, islands). After training on this data, the model reliably detected the horizon with high confidence. We used this to filter out background predictions from the original model, but unfortunately, it did not improve the mAP50 score.

2.3 Extra training data

To reduce overfitting and improve the models ability to detect objects in different lighting, angles and zoom we early on decided to expand the dataset as much as possible. With respect to the rules of only obtaining external data from public, free and open sources we figured out we could watch the series "Hurtigruten minutt for minutt" and take screenshots at moments boats appeared in the picture.

Before training the model with this data, we needed to annotate it. To do this we installed "labelImg", a program for highlighting and labeling objects in picture. We followed the same format as the given dataset and included the same labels. A source of error, is that we may not have annotated to the same standard as the handout dataset. For instance, we found that the handout data often ignored far-away vessels completely, while we were a bit more thorough. Another issue might have been that our interpretation of the vessel-classes could differ from that of the handout-annotators.

2.4 Compute Power

For model-training we mainly used the GPU-accelerator included with Kaggle and applied the GPU P100. Combined we consumed a total of 24.6MJ. We mainly used a batch size of 16, since we often ran out of GPU-memory with increased sizes. In retrospect, our energy use could have been better regulated, but due to a tight deadline we utilized all processing power available.

3 Reflections

All things considered our model performed decently. Our main issue was time management and looking back we probably should have researched studies similar to our scenario and figured out how to spend our time more efficiently. We spent a lot of time collecting and annotating data, data which resulted in being unusable. However, we did implement some ideas that worked out nicely, like image slicing and additional filtering.

If we had more time we probably would explored more models like DETR or R-CNN. We were never totally convinced the YOLO12m-model were most optimal. In early stages we chose to neglect the advantage of hyper parameters, so we never fully optimized the model at hand. Although the idea of an expanded dataset didn't work as planned it should help train the model and in future projects we would automate the annotation to save time. Overall the hackathon has been an enlightening experience, giving our group a solid introduction to machine learning.

References

- Akyon, Fatih Cagatay, Sinan Onur Altinuc, and Alptekin Temizel (2022). “Slicing Aided Hyper Inference and Fine-Tuning for Small Object Detection”. In: *2022 IEEE International Conference on Image Processing (ICIP)*, pp. 966–970. DOI: 10.1109/ICIP46576.2022.9897990.
- Khanam, Rahima and Muhammad Hussain (2024). *YOLOv11: An Overview of the Key Architectural Enhancements*. arXiv: 2410.17725 [cs.CV]. URL: <https://arxiv.org/abs/2410.17725>.
- Tian, Yunjie, Qixiang Ye, and David Doermann (2025). “YOLOv12: Attention-Centric Real-Time Object Detectors”. In: *arXiv preprint arXiv:2502.12524*.